

Programa de Pós-graduação em Políticas Públicas
Universidade Federal do ABC
PPU-009 - MÉTODOS QUANTITATIVOS

Docente: Professor Ricardo Ceneviva

Discentes: Debora Rodrigues Pereira Dias– Matrícula nº 21202510146

Manuel Mfinda Pedro Marques – Matrícula nº 21202510196

Desafio de Replicação: avaliando os dados e as evidências publicadas
Artigo escolhido: DESIGUALDADES SOCIOESPACIAIS DE ACESSO A
OPORTUNIDADES NAS CIDADES BRASILEIRAS – 2019 TD 2535

1. APÊNDICE COMPUTACIONAL - SCRIPT R

```
#-----#
#Desafio de Replicação:avaliando os dados e as evidências publicadas
#Métodos Quantitativos - Professor Ricardo Ceneviva
#Discentes: Debora Rodrigues Pereira Dias– Matrícula no 21202510146
#      Manuel Mfinda Pedro Marques – Matrícula no 21202510196
#DESIGUALDADES SOCIOESPACIAIS DE ACESSO A OPORTUNIDADES NAS CIDADES
BRASILEIRAS – 2019
# Artigo: TD 2535 - IPEA (2019) | Recorte: Curitiba/PR
#-----#

# Instala o "motor" de instalação
if (!require("remotes")) install.packages("remotes")

# Instala o aopdata diretamente do IPEA
remotes::install_github("ipeaGIT/aopdata", subdir = "r-package", upgrade = "never")

#Copie e cole este código (Carregar e Baixar Dados)
#Após a instalação terminar, cole este bloco para finalmente baixar os dados da pesquisa:

# Carregar o pacote
library(aopdata)

# 1. SETUP E CARREGAMENTO
library(aopdata)
```

```

library(ggplot2)
library(sf)
library(data.table)

# 2. Download dos dados
# Baixando dados de Curitiba para transporte público (2019)
dados_curitiba <- read_access(city='Curitiba', mode='public_transport', year=2019, geometry=TRUE)
setDT(dados_curitiba)
dados_sf <- st_as_sf(dados_curitiba)

# 3. REPLICAÇÃO (Verifiability - 60 min)
# Trabalho (Baseado no Mapa 5A do artigo original)
ggplot(data = dados_sf) + geom_sf(aes(fill = CMATT60), color = NA) +
  scale_fill_viridis_c(option = "viridis") + theme_minimal() +
  labs(title = "Replicação Mapa 5A: Trabalho (60 min)", fill = "Empregos")

# Escola (Mapa 5B)
ggplot(data = dados_sf) + geom_sf(aes(fill = CMAET60), color = NA) +
  scale_fill_viridis_c(option = "magma") + theme_minimal() +
  labs(title = "Replicação Mapa 5B: Educação (60 min)", fill = "Escolas")

# 4. TESTES DE ROBUSTEZ (Robustness - 30 min)
# Trabalho 30 min
ggplot(data = dados_sf) + geom_sf(aes(fill = CMATT30), color = NA) +
  scale_fill_viridis_c(option = "viridis") + theme_minimal() +
  labs(title = "Robustez Trabalho: Acesso em 30 min")

# Escola 30 min
ggplot(data = dados_sf) + geom_sf(aes(fill = CMAET30), color = NA) +
  scale_fill_viridis_c(option = "magma") + theme_minimal() +
  labs(title = "Robustez Escola: Acesso em 30 min")

# 5. ANÁLISE ESTATÍSTICA E GENERALIZAÇÃO (Palma Ratio)
# Cálculo ponderado pela população (P001) para os decis de renda (R003)
p_rico_esc <- dados_curitiba[R003 == 10, weighted.mean(CMAET60, P001, na.rm=T)]
p_pobre_esc <- dados_curitiba[R003 <= 4, weighted.mean(CMAET60, P001, na.rm=T)]
p_rico_trab <- dados_curitiba[R003 == 10, weighted.mean(CMATT60, P001, na.rm=T)]
p_pobre_trab <- dados_curitiba[R003 <= 4, weighted.mean(CMATT60, P001, na.rm=T)]

cat("--- RESULTADOS FINAIS ---\n")

```

```

cat("Palma Ratio Educação (60 min):", p_rico_esc / p_pobre_esc, "\n")
cat("Palma Ratio Trabalho (60 min):", p_rico_trab / p_pobre_trab, "\n")

# 6. EXIBIÇÃO DOS RESULTADOS (Saída de Verificação)
# -----
cat("\n===== \n")
cat("    RESULTADOS DA REPLICAÇÃO - CURITIBA (2019)    \n")
cat("===== \n")

# Imprime os valores do Palma Ratio
cat("\n>>> INDICADORES DE DESIGUALDADE (PALMA RATIO):\n")
cat("Acessibilidade Educacional (60 min):", round(p_rico_esc / p_pobre_esc, 2), "\n")
cat("Acessibilidade ao Trabalho (60 min):", round(p_rico_trab / p_pobre_trab, 2), "\n")

cat("\n>>> INTERPRETAÇÃO:\n")
cat("Valores acima de 1.0 indicam vantagem para o grupo de renda alta.\n")
cat("===== \n")

# 7. INFORMAÇÕES DO AMBIENTE (Reprodutibilidade T3)
# -----
# O comando abaixo é vital para o Marco T3 do Professor Ceneviva
print(sessionInfo())

```

2. Log de Execução e Validação de Ambiente (Output do Console)

```
> #-----#  
> #Desafio de Replicação:avaliando os dados e as evidências publica .... [TRUNCATED]
```

Carregando pacotes exigidos: remotes

```
> # Instala o aopdata diretamente do IPEA  
> remotes::install_github("ipeaGIT/aopdata", subdir = "r-package", upgrade = "never")
```

Using GitHub PAT from the git credential store.

Skipping install of 'aopdata' from a github remote, the SHA1 (478044ae) has not changed since last install.

Use `force = TRUE` to force installation

```
> #Copie e cole este código (Carregar e Baixar Dados)  
> #Após a instalação terminar, cole este bloco para finalmente baixar os dados da pesquisa:  
>  
> .... [TRUNCATED]
```

```
> # 1. SETUP E CARREGAMENTO  
> library(aopdata)
```

```
> library(ggplot2)
```

```
> library(sf)
```

Linking to GEOS 3.13.1, GDAL 3.11.4, PROJ 9.7.0; sf_use_s2() is TRUE

```
> library(data.table)
```

data.table 1.17.8 usando 4 threads (veja ?getDTthreads). Últimas notícias: r-datatable.com

Executando data.table em português; o suporte ao pacote está disponível apenas em inglês. Ao procurar ajuda online, certifique-se de verificar também a mensagem de erro em inglês. Isso pode ser obtido examinando os arquivos po/R-pt_BR.po e po/pt_BR.po no código-fonte do pacote, onde as mensagens de erro no idioma nativo e em inglês podem ser encontradas lado a lado. You can also try calling Sys.setLanguage('en') prior to reproducing the error message.

```
> # 2. Download dos dados  
> # Baixando dados de Curitiba para transporte público (2019)  
> dados_curitiba <- read_access(city='Curitiba', mode='public_ .... [TRUNCATED]
```

Using mode public_transport

Downloading accessibility data for the year 2019

Download status: 1 done; 0 in progress. Total size: 6.72 Mb (100%)... done!

Downloading land use data for the year 2019

Download status: 1 done; 0 in progress. Total size: 348.97 Kb (100%)... done!

Downloading population data for the year 2010

Download status: 1 done; 0 in progress. Total size: 404.55 Kb (100%)... done!

Download status: 1 done; 0 in progress. Total size: 1.25 Mb (361%)... done!

>

3. Dados do autores

```
#~~~~~  
~~~~~  
##### 0.1.1 Download de shape file de municipios e setores censitarios dos  
municipios incluidos no projeto  
# Esse script faz o download do shape dos municipios e setores censitarios do  
pacote geobr,  
# e salva-os na pasta de data-raw  
# Por mais que os shapes nao mudem entre os anos, eles sao salvos em pastas  
separadas para cada um dos anos  
# Isso em geral causa multiplicacao de shapes para a maioria das cidades, mas  
algumas cidades podem aprensetar
```

```
# mudanca no seu shape (goiania p/ 2019, por exemplo, q considera a RM) e  
portanto devem ser consideradas
```

```
# carregar bibliotecas  
source('./R/fun/setup.R')  
# função do geobr para dissolver polígonos  
source("R/fun/dissolve_polygons.R")
```

```
# 1. Funcao para download de shape file dos municipios e setores censitarios -----  
-----
```

```
download_muni_setores <- function(ano, munis = "all") {
```

```
  download_muni_setores_un <- function(sigla_muni) {
```

```
    # extract code muni  
    code_munis <- munis_list$munis_metro[abrev_muni == sigla_muni & ano_metro  
== ano]$code_muni %>%  
      unlist()
```

```
    # extract state code  
    state_code <- substr(code_munis[1], 1, 2) %>% as.numeric()
```

```
    # criar pasta do municipios  
    dir.create(sprintf("../data-raw/municipios/%s", ano))  
    dir.create(sprintf("../data-raw/setores_censitarios/%s", ano))
```

```
    # Download de arquivos - shapes dos municipios  
    uf_sf <- geobr::read_municipality(state_code)  
    muni_sf <- uf_sf %>% filter(code_muni %in% code_munis)  
    # muni_sf <- purrr::map_dfr(code_munis, geobr::read_municipality, year=ano,  
simplified = F)  
    # Dissolver os polígonos dos municípios componentes da RM  
    muni_sf <- dissolve_polygons(muni_sf, group_column = "code_state")  
    muni_sf <- st_transform(muni_sf, 4326)
```

```
    # Download de arquivos - shapes dos setores censitarios  
    uf_sf_tracts <- geobr::read_census_tract(state_code, simplified = FALSE)  
    ct_sf <- uf_sf_tracts %>% filter(code_muni %in% code_munis)  
    ct_sf <- st_transform(ct_sf, 4326)
```

```

# mapview(ct_sf)

# salvar municipios
readr::write_rds(muni_sf, sprintf("../data/municipios/%s/municipio_%s_%s.rds",
ano, sigla_muni, ano), compress = 'gz')

# salvar setores censitarios
readr::write_rds(ct_sf,
sprintf("../data/setores_censitarios/%s/setores_%s_%s.rds", ano, sigla_muni, ano),
compress = 'gz')
}

# 2. Aplica funcao -----
if (munis == "all") {

# seleciona todos municipios ou RMs do ano escolhido
x = munis_list$munis_metro[ano_metro == ano]$abrev_muni

} else (x = munis)

lapply(X=x, FUN=download_muni_setores_un)

}

download_muni_setores(ano = 2017)
download_muni_setores(ano = 2018)
download_muni_setores(ano = 2019)
download_muni_setores(ano = 2020)

#~~~~~
~~~~~
##### 0.1.5 Dados do CNES
# Esse script aplica funcoes que foram definidas na pasta R/fun/saude p/ todos os
anos de processamento
# Sao tres funcoes principais:
# 1) Filtra cada base de saude mantendo somente observacoes de equipamentos de
saude publicos e e municipios do projeto
# 2) Traz o geocode que foi realizado com o streetmap e aprimora a qualidade do
geocode quando necessario
# 3) Filtra somente as observacoes que apresentam boa qualidade de geocode

```

```

# carregar bibliotecas
source('./R/fun/setup.R')

# carregar funcoes
source('./R/fun/saude/saude.R')

# Aplicar funcao filtro
saude_filter(2017)
saude_filter(2018)
saude_filter(2019)

# Aplicar funcao geocode
saude_geocode(2017)
saude_geocode(2018)
saude_geocode(2019)

# Selecionar somente as observacoes com boa qualidade de geocode
source("R/fun/filter_geocode.R")
lapply(X=2017:2019, geocode_filter, "saude")

#~~~~~
~~~~~
##### 0.1.4 Dados do censo escolar
# Esse script aplica funcoes que foram definidas na pasta R/fun/educacao p/ todos
os anos de processamento
# Sao tres funcoes principais:
# 1) Filtra cada base de educacao mantendo somente observacoes de escolas
publicas e e municipios do projeto
# 2) Traz o geocode que foi realizado com o streetmap e aprimora a qualidade do
geocode quando necessario
# 3) Filtra somente as observacoes que apresentam boa qualidade de geocode

# carregar bibliotecas
source('./R/fun/setup.R')

# carregar funcoes
source('./R/fun/educacao/educacao.R')

# 1) Aplicar funcao filtro -----
lapply(X=2017:2019, FUN=educacao_filter)

# 2) Aplicar funcao geocode -----

```



```

educacao_geocode(run_gmaps = FALSE)

# 3) Selecionar somente as obssecoes com boa qualidade de geocode -----
source("R/fun/filter_geocode.R")
lapply(X=2017:2019, geocode_filter, "educacao")

#-----
##### 0.1.6 Limpeza e Geocode dados da RAIS
# Esse script aplica funcoes que foram definidas na pasta R/fun/empregos p/ todos
os anos de processamento
# Sao as funcoes principais:
# 0) Save raw data with columns we use in the municipalities of the project
# 1) Filter raw workers data - only private jobs
# 2) Categorizar trabalhadores por grau de instrucao
# 3) Tratar os outliers
# 4) Trazer o geocode dos estabelecimentos e aprimorar a qualidade quando
necessario
# 5) Trazer geocode p/ a base original (da etapa 2)
# 6) Selecionar somente as obssecoes com boa qualidade de geocode

# carregar bibliotecas
source('./R/fun/setup.R')

# carregar funcoes
purrr::walk(dir('./R/fun/empregos', full.names = TRUE), source)

# Aplicar funcoes para o ano de 2017 -----

# 0) Save raw data with columns we use in the municipalities of the project
rais_filter_raw_data(2017)

# 1) Filter raw workers data - only private jobs
rais_filter_pessoas(2017)

# 2) Categorizar trabalhadores por grau de instrucao
rais_categorize_inst(2017)

# 3) Tratar os outliers
rais_treat_outliers(2017)

# 4) Aprimorar o geocode dos estabelecimentos que veio do streetmap

```

```
# primeiro, limpar a base completa geocoded que veio do streetmap
rais_clean_estabs_raw(2017)
# segundo, aprimorar o geocde utilizando o geocode do gmaps
rais_gmaps_geocode(2017, run_gmaps = FALSE)

# 5) Trazer geocode p/ a base original (da etapa 2)
rais_bring_geocode(2017)

# 6) Selecionar somente as obssecoes com boa qualidade de geocode
source("R/fun/filter_geocode.R")
geocode_filter(ano = 2017, "rais")


# Aplicar funcoes para o ano de 2018 -----

# 0) Save raw data with columns we use in the municipalities of the project
rais_filter_raw_data(2018)

# 1) Filter raw workers data - only private jobs
rais_filter_pessoas(2018)

# 2) Categorizar trabalhadores por grau de instrucao
rais_categorize_inst(2018)

# 3) Tratar os outliers
rais_treat_outliers(2018)

# 4) Aprimorar o geocode dos estabelecimentos que veio do streetmap
# primeiro, limpar a base completa geocoded que veio do streetmap
rais_clean_estabs_raw(2018)
# segundo, aprimorar o geocde utilizando o geocode do gmaps
rais_gmaps_geocode(2018, run_gmaps = FALSE)

# 5) Trazer geocode p/ a base original (da etapa 2)
rais_bring_geocode(2018)

# 6) Selecionar somente as obssecoes com boa qualidade de geocode
source("R/fun/filter_geocode.R")
geocode_filter(ano = 2018, "rais")
```

```

# Aplicar funcoes para o ano de 2019 -----

# 0) Save raw data with columns we use in the municipalities of the project
rais_filter_raw_data(2019)

# 1) Filter raw workers data - only private jobs
rais_filter_pessoas(2019)

# 2) Categorizar trabalhadores por grau de instrucao
rais_categorize_inst(2019)

# 3) Tratar os outliers
rais_treat_outliers(2019)

# 4) Aprimorar o geocode dos estabelecimentos que veio do streetmap
# primeiro, limpar a base completa geocoded que veio do streetmap
rais_clean_estabs_raw(2019)
# segundo, aprimorar o geocde utilizando o geocode do gmaps
rais_gmaps_geocode(2019, run_gmaps = FALSE)

# 5) Trazer geocode p/ a base original (da etapa 2)
rais_bring_geocode(2019)

# 6) Selecionar somente as obssecoes com boa qualidade de geocode
source("R/fun/filter_geocode.R")
geocode_filter(ano = 2019, "rais")

#~~~~~
~~~~~
##### Leitura e filtro de elevacao/topografia

#' info:
#' Os dados de topografia são provenientes da missão SRTM (_Shuttle Radar
#' Topography Mission_), que é um esforço de pesquisa internacional que obtém
#' dados de elevação numa precisão de 30 metros. Os dados de elevação do SRTM
são
#' divididos por quadrículo de 1 grau de latitude e 1 longitude, então é
#' necessário:
#' 1 - identificar os quadrículos que cobrem a cidade escolhida;
#' 2 - baixar o raster correspondente a cada quadrículos em uma pasta temporária;
#' 3 - unir os grids em um grid único, usando a função raster::mosaic();
#' 4 - recortar do raster a área correspondente ao bounding box do município;
#' 5 - salvar o raster do município na pasta correspondente.

```

```

#
# Para ter acesso a esses dados, é necessário criar um login no site
# https://urs.earthdata.nasa.gov, e informar os dados de usuário e senha quando
# for rodar esse script.

# carregar bibliotecas
source('./R/fun/setup.R')
library(httr)
library(raster)

message("Inform username and password to access https://urs.earthdata.nasa.gov
")
username <- readline("Give the username : ")
password <- readline("Give the password : ")

# username <- "user"
# password <- "pass"

# ano <- 2020
# sigla_muni <- "man"
# sigla_muni="for"

download_srtm <- function(sigla_muni) {
  # read municipality boundary
  muni_sf <- readr::read_rds(sprintf("../data-
raw/municipios/2020/municipio_%s_2020.rds", sigla_muni))

  # muni_sf %>% mapview()

  # extract bounding box
  bbox <- st_bbox(muni_sf)
  bbox <- as.integer(bbox) - 1

  # identify which tiles are needed to cover the whole study area
  lons <- seq(floor(bbox[1]), ceiling(bbox[3]), by = 1)
  lats <- seq(floor(bbox[2]), ceiling(bbox[4]), by = 1)
  tiles <- expand.grid(lat = lats, lon = lons) %>%
    mutate(hx = if_else(lon < 0, "W", "E"),
           hy = if_else(lat < 0, "S", "N"))
  tile = sprintf("%s%02d%s%03d", tiles$hy, abs(tiles$lat), tiles$hx, abs(tiles$lon))

  # build the url's for each tile

```

```

urls <-
paste0("https://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/",
      tile, ".SRTMGL1.hgt.zip")

# download zip files and extract raster tiles
outputdir <- tempdir()
zipfiles <- paste0(outputdir, "\\ ", tile, ".hgt.zip")
rstfiles <- paste0(outputdir, "\\ ", tile, ".hgt")

walk2(urls, zipfiles, function(url, filename) {
  http::GET(url = url,
            authenticate(username, password),
            write_disk(path = filename, overwrite = TRUE),
            progress())
})

walk(zipfiles, unzip, exdir = outputdir)

# read all raster tiles, merge them together, and then crop to the study area's
# bounding box
rst <- map(rstfiles, raster)
if (length(rst) == 1) {
  rst_layer <- rst[[1]]
} else {
  rst_layer <- do.call(raster::mosaic, args = c(rst, fun = mean))
}
rst_layer_crop <- raster::crop(rst_layer, st_bbox(muni_sf))

# save processed raster to the municipality folder
dir.create(paste0("../data/acesso_oport/topografia/", sigla_muni, "/"), recursive =
TRUE)
raster::writeRaster(rst_layer_crop,
                  paste0("../data/acesso_oport/topografia/", sigla_muni, "/topografia_",
sigla_muni, ".tif"),
                  overwrite = TRUE)
}

# download_srtm("poa")
# download_srtm("bel")
walk(munis_list$munis_df$abrev_muni, download_srtm)

```

Instalação

Você pode instalar pelo CRAN ou pelo desenvolvimento versão do GitHub.aopdata

```
# CRAN
```

```
install.packages("aopdata")
```

```
# dev version from github
```

```
utils::remove.packages('aopdata')
```

```
devtools::install_github("ipeaGIT/aopdata", subdir = "r-package")
```

Uso Básico

Primeiro, você precisa carregar o pacote.

```
library(aopdata)
```

Dicionário de dados

O dicionário de colunas de dados é apresentado na documentação de cada função. No entanto, você também pode abrir o dicionário de dados na web navegador executando:

```
# for English
```

```
aopdata_dictionary(lang = 'en')
```

```
# for Portuguese
```

```
aopdata_dictionary(lang = 'pt')
```

Estimativas de acessibilidade

A função baixa acessibilidade estimativas para um dado , e . Para maior conveniência, essa função também será baixe automaticamente os dados de população e uso do solo das cidades Selecionado. Note que estimativas de acessibilidade estão disponíveis para picos e Períodos fora do pico para modos e E.[read_access\(\)](#)citymodeyearpublic_transportcar

```
# Download accessibility, population and land use data
```

```
df <- aopdata::read_access(  
  city = 'Curitiba',  
  mode = 'public_transport',  
  peak = TRUE,  
  year = 2019,  
  showProgress = FALSE  
)
```

Você também pode definir o parâmetro para que funções retornam um objeto espacial com as geometrias de a grade espacial H3.`geometry = TRUEsf`

Download accessibility, population and land use data

```
df <- aopdata::read_access(  
  city = 'Curitiba',  
  mode = 'public_transport',  
  peak = TRUE,  
  year = 2019,  
  geometry = TRUE  
)
```

Dados de população e uso do solo

Caso você esteja interessado apenas em usar a população e o uso do solo dados gerados pelo Projeto Access to Opportunities, você pode baixar esses conjuntos de dados separadamente. Por favor, note que a população disponível vem do mais recente censo brasileiro de 2010, enquanto os dados de uso do solo podem ser baixado para 2017, 2018 ou 2019.

Land use data

```
df <- aopdata::read_landuse(  
  city = 'Fortaleza',
```

```
year = 2019,  
geometry = TRUE,  
showProgress = FALSE  
)
```

Population data

```
df <- aopdata::read_population(  
  city = 'Fortaleza',  
  year = 2010,  
  geometry = TRUE,  
  showProgress = FALSE  
)
```

Leia apenas dados de grade espacial

Caso você queira baixar apenas a grade espacial H3 de cidades no projeto AOP, você pode usar a função [read_grid\(\)](#)

```
h3 <- aopdata::read_grid(city = 'Fortaleza', showProgress = FALSE)
```

Nota

Em todas as funções acima, note que:

- O parâmetro também pode ser uma abreviação de 3 letras da cidade.city

```
df <- aopdata::read_access(  
  city = 'cur',  
  mode = 'public_transport',  
  year = 2019,  
  peak = TRUE,
```



```
showProgress = FALSE
```

```
)
```

```
df <- aopdata::read_grid(city = 'for', showProgress = FALSE)
```

- Você também pode baixar os dados de todas as cidades do projeto ao mesmo tempo Usando :city = 'all'

```
df <- aopdata::read_landuse(city = 'all', year = 2019)
```

Mapeamento da acessibilidade urbana

Carregar bibliotecas

```
library(aopdata)
```

```
library(sf)
```

```
library(ggplot2)
```

```
library(data.table)
```

```
library(scales)
```

Baixar dados de acessibilidade

```
# download aop data
```

```
df <- aopdata::read_access(
```

```
  city = 'Curitiba',
```

```
  mode = 'public_transport',
```

```
  year = 2019,
```

```
  peak = FALSE,
```

```
  geometry = TRUE,
```

```
  showProgress = FALSE
```

```
)
```

Mapa o acesso às oportunidades de emprego

```
ggplot() +
```

```
  geom_sf(data=df, aes(fill=CMATT60), color=NA, alpha=.9) +
```

```
  scale_fill_viridis_c(option = "inferno", labels = scales::comma) +
```

```
  labs(title='Number of jobs accessible', fill="Accessibility",
```

```
        subtitle='by public transport in less than 60 min.') +
```

```
  theme_void()
```

Mapeamento de dados populacionais

Aqui estão alguns exemplos rápidos para ilustrar como você pode usar o {aopdata} para mapear a distribuição espacial da população em Cidades brasileiras.

```
# load libraries
library(aopdata)
library(data.table)
library(ggplot2)
library(sf)
library(scales)
```

Baixar dados populacionais

```
df <- aopdata::read_population(
  city = 'Fortaleza',
  year = 2010,
  geometry = TRUE,
  showProgress = FALSE
)
```

Mapa populacional total

```
ggplot() +
  geom_sf(data=subset(df, P001>0), aes(fill=P001), color=NA, alpha=.8) +
  scale_fill_distiller(palette = "YlOrRd", direction = 1)+
  labs(title='Population distribution', fill="Total population") +
  theme_void()
```

Mapeamento populacional por níveis de renda

Aqui, mapeamos a distribuição espacial da população por decil de renda (coluna).R003

```
ggplot() +
  geom_sf(data=subset(df, !is.na(R002)), aes(fill=factor(R003)), color=NA, alpha=.8)
+
  scale_fill_brewer(palette = "RdBu") +
  labs(title='Average household income per capita', fill="Income decile") +
  theme_void()
```

População do mapa por raça

Aqui, mapeamos a distribuição espacial da população negra.

```
df$prop_black <- df$P003 / df$P001
```

```
ggplot() +  
  geom_sf(data=subset(df, P001 >0), aes(fill=prop_black), color=NA, alpha=.8) +  
  scale_fill_distiller(palette = "RdPu", direction = 1, labels = percent)+  
  labs(title='Proportion of black population', fill="Black population") +  
  theme_void()
```

Maapeamento dos dados de uso do solo

Aqui estão alguns exemplos rápidos para ilustrar como você pode usar o {aopdata} para mapear a distribuição espacial das atividades e serviços urbanos em cidades brasileiras.

```
# load libraries  
library(aopdata)  
library(data.table)  
library(ggplot2)  
library(sf)  
library(scales)
```

Baixar dados de uso do solo

```
df <- aopdata::read_landuse(  
  city = 'Fortaleza',  
  year = 2019,  
  geometry = T,  
  showProgress = F  
)
```

Distribuição espacial dos empregos

```
ggplot() +  
  geom_sf(data=df, aes(fill=T001), color=NA, alpha=.9) +  
  scale_fill_distiller(palette = "YlOrRd", direction = 1) +  
  labs(title='Spatial distribution of jobs', fill="N. of jobs") +  
  theme_void()
```

Distribuição espacial das escolas

Neste caso, abaixo, escolas primárias com o coluna.E003

```
ggplot() +  
  geom_sf(data=df, aes(fill=factor(E003)), color=NA, alpha=.9) +  
  scale_fill_brewer(palette = "PuBuGn", direction = 1) +  
  labs(title='Spatial distribution of elementary schools', fill="N. of schools")  
+  
  theme_void()
```

Distribuição espacial da saúde

Neste exemplo, mapeamos instalações de saúde de alta complexidade (coluna).S004

```
ggplot() +  
  geom_sf(data=df, aes(fill=factor(S004)), color=NA, alpha=.9) +  
  scale_fill_brewer(palette = "YlGnBu", direction = 1)+  
  labs(title='Spatial distribution of hospitals', fill="N. of hospitals") +  
  theme_void()
```

Centros de Assistência Social do Mapa (CRAS)

```
ggplot() +  
  geom_sf(data=df, aes(fill=factor(C001)), color=NA, alpha=.9) +  
  scale_fill_brewer(palette = "RdPu", direction = 1)+  
  labs(title='Spatial distribution of CRAS facilities', fill="N. of CRAS") +  
  theme_void()
```

Analizando a desigualdade no acesso a oportunidades

Desigualdades sociais e raciais no acesso às oportunidades

```
# load libraries  
library(aopdata)  
library(sf)  
library(ggplot2)  
library(data.table)  
library(units)
```

Baixar dados de acessibilidade

```
df <- aopdata::read_access(  
  city = 'Curitiba',  
  mode = 'public_transport',  
  peak = TRUE,  
  year = 2019,  
  showProgress = FALSE  
)
```

Desigualdade no acesso a oportunidades de emprego por decil de renda

```
ggplot() +  
  geom_boxplot(data=subset(df, !is.na(R003)),  
    aes(x = factor(R003), y=CMATT60/1000, color=factor(R003))) +  
  scale_color_brewer(palette = 'RdBu') +  
  labs(title='Distribution of the number of jobs accessible',  
    color="Income\ndecile",  
    subtitle='by public transport in less than 60 min. by income decile',  
    x='Income decile', y="N. of jobs accessible\n(thousands)") +  
  scale_x_discrete(labels=c("D1 Poorest", paste0('D', 2:9), "D10 Wealthiest"))  
+  
  theme_minimal()
```

Razão de Palma

```
# average access of the wealthiest 10%  
avg_access_10p_wealthiest <- df[ R003==10, weighted.mean(x=CMATT60,  
w=P001, na.rm=T)]
```

```
# average access of the poorest 40%  
avg_access_40p_poorest <- df[ R003<=4, weighted.mean(x=CMATT60,  
w=P001, na.rm=T)]
```

```
# Palma ratio
```

```
palma_ratio <- avg_access_10p_wealthiest / avg_access_40p_poorest  
palma_ratio
```

Isso significa que a população 10% mais rica poderia acessar pelo público transporte, em média, tem 2,6 vezes mais oportunidades de emprego do que os 40% As pessoas mais pobres em menos de 60 minutos.

Desigualdade no tempo de viagem para fechar hospital

```
# replace Inf travel time with 120
df[, TMISA := fifelse(TMISA==Inf, 120, TMISA)]

# calculate average travel time by race
df[, .(average = weighted.mean(x=TMISA, w=P001, na.rm=T),
      white   = weighted.mean(x=TMISA, w=P002, na.rm=T),
      black   = weighted.mean(x=TMISA, w=P003, na.rm=T))]

# calculate average travel time by income
temp <- df[, .(average = weighted.mean(x=TMISA, w=P001, na.rm=T)),
by=R003]
temp <- na.omit(temp)

ggplot() +
  geom_point(data=temp, aes(y=average, x=factor(R003))) +
  labs(x='Income decile', y='Avg. travel time to\nclosest hospital') +
  theme_minimal()
```